

Tecnologias Específicas em Arquitetura de Software

1. Docker & Containerização

1.1 O que é Docker?

Docker é uma plataforma de containerização que empacota aplicação, dependências e configuração em uma unidade isolada chamada container.

1.2 Conceitos Fundamentais

Imagem Docker

Modelo imutável que define como criar um container.

Plain Text

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Container

Instância em execução de uma imagem Docker.

Bash

```
docker run -p 8000:8000 minha-aplicacao:1.0
```

Registry

Repositório de imagens Docker (Docker Hub, ECR, etc).

Bash

```
docker push meu-registry/minha-aplicacao:1.0
```

1.3 Benefícios

- **Consistência:** Mesma imagem em dev, test, prod
- **Isolamento:** Aplicações não interferem umas nas outras
- **Portabilidade:** Funciona em qualquer lugar
- **Eficiência:** Menos overhead que VMs

1.4 Desvantagens

- **Curva de Aprendizado:** Conceitos novos
 - **Overhead de Rede:** Comunicação entre containers
 - **Armazenamento:** Múltiplas imagens ocupam espaço
 - **Segurança:** Requer configuração adequada
-

2. Kubernetes (K8s)

2.1 Definição

Kubernetes é um orquestrador de containers que automatiza deployment, escalabilidade e gerenciamento de aplicações containerizadas.

2.2 Conceitos Principais

Pod

Unidade básica do Kubernetes. Geralmente contém um container.

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: meu-app
spec:
  containers:
    - name: app
      image: minha-aplicacao:1.0
      ports:
        - containerPort: 8000
```

Deployment

Define como criar e atualizar Pods.

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: meu-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: meu-app
  template:
    metadata:
      labels:
        app: meu-app
    spec:
      containers:
        - name: app
          image: minha-aplicacao:1.0
```

Service

Expõe Pods para comunicação interna/externa.

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: meu-app-service
spec:
  selector:
    app: meu-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: LoadBalancer
```

ConfigMap

Armazena configurações não-sensíveis.

YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DATABASE_HOST: "postgres.default.svc.cluster.local"
  LOG_LEVEL: "INFO"
```

Secret

Armazena dados sensíveis (senhas, tokens).

YAML

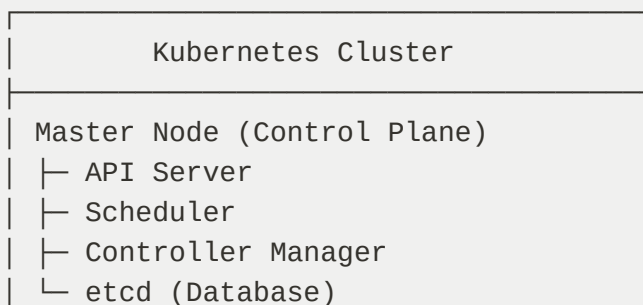
```
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
type: Opaque
data:
  DATABASE_PASSWORD: cGFzc3dvcmQxMjM= # base64
  API_KEY: YWJjZGVmZ2hpams= # base64
```

2.3 Recursos Gerenciados

- **Replicação:** Múltiplas instâncias de Pods
- **Escalabilidade:** Aumentar/diminuir Pods automaticamente
- **Self-Healing:** Reinicia Pods que falharam
- **Rolling Updates:** Atualiza Pods sem downtime
- **Load Balancing:** Distribui tráfego entre Pods

2.4 Arquitetura

Plain Text



```
Worker Node 1
├─ Pod 1 (Container)
├─ Pod 2 (Container)
└─ Kubelet, kube-proxy
```

```
Worker Node 2
├─ Pod 3 (Container)
├─ Pod 4 (Container)
└─ Kubelet, kube-proxy
```

3. Apache Kafka

3.1 Definição

Kafka é uma plataforma de streaming distribuída que permite publicar e subscrever em fluxos de eventos em tempo real.

3.2 Conceitos Fundamentais

Topic

Canal de comunicação para eventos.

Plain Text

```
Topic: "user-events"
├─ Partition 0: [Event1, Event2, Event3]
├─ Partition 1: [Event4, Event5, Event6]
└─ Partition 2: [Event7, Event8, Event9]
```

Producer

Publica eventos em um topic.

Python

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
```

```
)  
  
event = {"user_id": 123, "action": "login"}  
producer.send('user-events', value=event)
```

Consumer

Consome eventos de um topic.

Python

```
from kafka import KafkaConsumer  
import json  
  
consumer = KafkaConsumer(  
    'user-events',  
    bootstrap_servers=['localhost:9092'],  
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),  
    group_id='my-group'  
)  
  
for message in consumer:  
    print(f"Received: {message.value}")
```

Consumer Group

Grupo de consumers que compartilham carga.

Plain Text

Topic: "user-events" (3 partitions)

Consumer Group 1:

```
├─ Consumer 1 → Partition 0  
├─ Consumer 2 → Partition 1  
└─ Consumer 3 → Partition 2
```

3.3 Características

- **Durabilidade:** Eventos são persistidos em disco
- **Escalabilidade:** Partições permitem paralelismo
- **Replicação:** Múltiplas cópias para tolerância a falhas
- **Retenção:** Mantém histórico de eventos

- **Offset:** Rastreia posição de leitura

3.4 Casos de Uso

- **Event Streaming:** Fluxo contínuo de eventos
 - **Log Agregação:** Centralizar logs de múltiplas aplicações
 - **Análise Real-time:** Processar eventos conforme chegam
 - **Integração de Sistemas:** Conectar sistemas heterogêneos
-

4. APIs: REST vs gRPC vs GraphQL

4.1 REST (Representational State Transfer)

Características

- **HTTP Methods:** GET, POST, PUT, DELETE
- **Resources:** Tudo é um recurso com URL
- **Stateless:** Cada requisição é independente
- **Caching:** Suporte nativo a cache

Exemplo

Plain Text

```
GET /api/users/123
Content-Type: application/json
```

```
{
  "id": 123,
  "name": "João",
  "email": "joao@email.com"
}
```

Prós

- Simples de entender e usar
- Suporte nativo em browsers
- Caching eficiente
- Amplamente adotado

Contras

- Over-fetching: Retorna mais dados que necessário
- Under-fetching: Precisa fazer múltiplas requisições
- Versionamento complexo
- Sem tipagem forte

4.2 gRPC (gRPC Remote Procedure Call)

Características

- **HTTP/2**: Multiplexing e compressão
- **Protocol Buffers**: Serialização binária
- **Tipagem Forte**: Schemas definidos
- **Bidirecional**: Streaming em ambas direções

Exemplo

Plain Text

```
syntax = "proto3";

service UserService {
  rpc GetUser (GetUserRequest) returns (User);
  rpc ListUsers (Empty) returns (stream User);
}

message GetUserRequest {
  int32 id = 1;
}

message User {
  int32 id = 1;
  string name = 2;
  string email = 3;
}
```

Prós

- Muito rápido (binário vs JSON)
- Tipagem forte

- Streaming bidirecional
- Multiplexing HTTP/2

Contras

- Curva de aprendizado maior
- Não funciona bem em browsers
- Debugging mais difícil
- Menos ferramentas

4.3 GraphQL

Características

- **Query Language:** Cliente especifica dados necessários
- **Single Endpoint:** Uma URL para todas as operações
- **Tipagem Forte:** Schema define tipos
- **Sem Over/Under-fetching:** Retorna exatamente o necessário

Exemplo

Plain Text

```
query GetUser {  
  user(id: 123) {  
    id  
    name  
    email  
    posts {  
      id  
      title  
    }  
  }  
}  
  
response:  
{  
  "data": {  
    "user": {  
      "id": 123,  
      "name": "João",  
      "email": "joao@email.com",  
      "posts": [  

```

```
{
  "posts": [
    {"id": 1, "title": "Post 1"},
    {"id": 2, "title": "Post 2"}
  ]
}
```

Prós

- Cliente especifica dados necessários
- Sem over/under-fetching
- Tipagem forte
- Excelente para mobile

Contras

- Mais complexo de implementar
- Caching mais difícil
- N+1 queries problem
- Curva de aprendizado

4.4 Comparação

Aspecto	REST	gRPC	GraphQL
Performance	Média	Excelente	Boa
Simplicidade	Simples	Complexo	Médio
Tipagem	Não	Sim	Sim
Caching	Fácil	Difícil	Difícil
Streaming	Não	Sim	Não
Browser	Sim	Não	Sim
Adoção	Muito Alta	Crescente	Crescente

5. Message Brokers: RabbitMQ vs Kafka vs Pulsar

5.1 RabbitMQ

Características

- **AMQP Protocol:** Protocolo padrão
- **Queues:** Armazena mensagens em filas
- **Exchanges:** Roteia mensagens para filas
- **Confirmação:** Garante entrega

Arquitetura

Plain Text

Producer → Exchange → Queue → Consumer

↓

Dead Letter Queue (se falhar)

Prós

- Fácil de usar
- Confiável
- Suporte a múltiplos protocolos
- Bom para transações

Contras

- Menos escalável que Kafka
- Sem persistência de longa duração
- Overhead de memória

5.2 Apache Kafka

Características

- **Topics e Partitions:** Paralelismo
- **Consumer Groups:** Múltiplos consumers
- **Retenção:** Mantém histórico
- **Distribuído:** Múltiplos brokers

Prós

- Altamente escalável
- Retenção de longa duração
- Replay de eventos
- Throughput muito alto

Contras

- Mais complexo de operar
- Eventual consistency
- Overhead de armazenamento

5.3 Apache Pulsar

Características

- **Multi-tenancy:** Isolamento de tenants
- **Geo-replication:** Replicação entre regiões
- **Tiered Storage:** Armazena em diferentes camadas
- **Arquitetura Separada:** Brokers e storage separados

Prós

- Muito escalável
- Geo-replication nativo
- Tiered storage
- Melhor que Kafka em alguns aspectos

Contras

- Menos adotado que Kafka
- Comunidade menor
- Mais complexo

5.4 Comparação

Aspecto	RabbitMQ	Kafka	Pulsar
---------	----------	-------	--------

Escalabilidade	Média	Excelente	Excelente
Retenção	Curta	Longa	Longa
Throughput	Médio	Muito Alto	Muito Alto
Complexidade	Baixa	Média	Alta
Adoção	Alta	Muito Alta	Crescente
Geo-replication	Não	Complexo	Nativo

6. Observabilidade: Ferramentas Práticas

6.1 Prometheus

Coleta de métricas time-series.

YAML

```
# prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'api-server'
    static_configs:
      - targets: ['localhost:8000']
```

Python

```
# Código da aplicação
from prometheus_client import Counter, Histogram

request_count = Counter('requests_total', 'Total requests')
request_duration = Histogram('request_duration_seconds', 'Request duration')

@request_duration.time()
def handle_request():
    request_count.inc()
    # ... lógica da requisição
```

6.2 Grafana

Visualização de métricas.

Plain Text

```
Dashboard: API Performance
├─ Graph: Requests/sec
├─ Graph: Error Rate
├─ Gauge: CPU Usage
└─ Heatmap: Latency Distribution
```

6.3 ELK Stack (Elasticsearch, Logstash, Kibana)

Logging centralizado.

Plain Text

```
Aplicação → Logstash → Elasticsearch → Kibana
  (logs)      (processa)  (armazena)      (visualiza)
```

6.4 Jaeger

Distributed tracing.

Plain Text

```
Request → Service A (10ms)
        → Service B (25ms)
        → Service C (15ms)
Total: 50ms
```

Jaeger mostra:

- Tempo em cada serviço
- Chamadas entre serviços
- Erros em cada etapa

7. Infrastructure as Code (IaC)

7.1 Terraform

Define infraestrutura como código.

Plain Text

```

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfafa1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
}

resource "aws_db_instance" "postgres" {
  allocated_storage = 20
  engine            = "postgres"
  engine_version    = "13.7"
  instance_class     = "db.t2.micro"
  name              = "mydb"
  username          = "admin"
  password          = var.db_password
  skip_final_snapshot = true
}

```

7.2 Ansible

Automação de configuração.

YAML

```

---
- hosts: web_servers
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Copy config file
      copy:
        src: nginx.conf

```

```
dest: /etc/nginx/nginx.conf
notify: restart nginx
```

7.3 CloudFormation

Infraestrutura na AWS.

YAML

```
AWSTemplateFormatVersion: '2010-09-09'
Resources:
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0c55b159cbfafa1f0
      InstanceType: t2.micro
      Tags:
        - Key: Name
          Value: web-server
```

8. CI/CD Pipelines

8.1 GitLab CI/CD

YAML

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - docker build -t minha-app:$CI_COMMIT_SHA .
    - docker push registry.gitlab.com/minha-app:$CI_COMMIT_SHA

test:
  stage: test
  script:
    - pytest tests/
    - coverage report

deploy:
```



```
stage: deploy
script:
  - kubectl set image deployment/minha-app app=registry.gitlab.com/minha-app:
only:
  - main
```

8.2 GitHub Actions

YAML

```
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Build Docker image
        run: docker build -t minha-app:${{ github.sha }} .

      - name: Run tests
        run: docker run minha-app:${{ github.sha }} pytest

      - name: Push to registry
        run: docker push registry.example.com/minha-app:${{ github.sha }}
```

9. Monitoramento de Aplicação (APM)

9.1 New Relic

Monitoramento de performance de aplicação.

Python

```
import newrelic.agent

newrelic.agent.initialize('newrelic.ini')
```

```
@newrelic.agent.function_trace()
def process_order(order_id):
    # ... lógica
    pass
```

9.2 Datadog

Observabilidade completa.

Python

```
from datadog import initialize, api

options = {
    'api_key': 'YOUR_API_KEY',
    'app_key': 'YOUR_APP_KEY'
}

initialize(**options)

# Enviar métrica customizada
api.Metric.send(
    metric='custom.order.processing_time',
    points=[(time.time(), 150)], # 150ms
    tags=['service:order-processor']
)
```

Referências

- Docker Documentation: <https://docs.docker.com/>
- Kubernetes Documentation: <https://kubernetes.io/docs/>
- Apache Kafka Documentation: <https://kafka.apache.org/documentation/>
- Prometheus Documentation: <https://prometheus.io/docs/>
- Terraform Documentation: <https://www.terraform.io/docs/>

Documento preparado por: Manus AI

Data: Maio 2026

Versão: 1.0